

Feed

The Groww Feed provides methods to subscribe to and receive Groww data streams and updates.

Feed Client

Once you have your [API key](#), you can use the `GrowFeed` client to subscribe to live data streams and receive updates.

The Feed Client can either be used synchronously to get the last updated data or asynchronously to trigger a callback whenever new data is received.

To use the methods in Feed Client, you need an exchange token for the corresponding instrument. You can get the exchange token of a particular instrument from the [instruments csv file](#).

You can subscribe for up to 1000 instruments at a time.

Feed Methods

The SDK provides methods to subscribe to and receive live data streams using the `GrowFeed` class accessible from the `growapi` module.

1. Live Data: Subscribe to, get, and unsubscribe from the live data of Derivatives, Stocks and Indices. Each of them support subscribing, getting data and unsubscribing for multiple instruments in one go.
2. Order updates of Derivatives and Equity and also Position updates of Derivatives.

Live Data

Subscribe and get the live data of equity, index and derivatives.

Live Data for Equity and Derivatives

Subscribe to, get, and unsubscribe from the live data for equities or derivatives contracts.

```
# Grow API Credentials (Replace with your actual credentials)
API_AUTH_TOKEN = "your_token"

grow = GrowAPI(API_AUTH_TOKEN)
feed = GrowFeed(grow)

def on_data_received(meta): # callback function which gets triggered when data is received
    print("Data received")
    print(feed.get_ltp())

# you can fetch exchange_token from instruments.csv file
instruments_list = [{"exchange": "NSE", "segment": "CASH", "exchange_token": "2885"}, {"exchange": "NSE", "segment": "FUT", "exchange_token": "2885"}]

feed.subscribe_ltp(instruments_list, on_data_received)

# This is a blocking call. Nothing after this will run.
feed.consume()

# OR

# you can also fetch data synchronously
feed.subscribe_ltp(instruments_list)

# Live data can also be continuously polled using this method
for i in range(10):
    time.sleep(1)
    print(feed.get_ltp())

feed.unsubscribe_ltp(instruments_list)
```

Live Data for Equity and Derivatives - Request Fields

Name	Type	Description
exchange	str	<u>Stock Exchange</u>
segment	str	<u>Segment</u> of the instrument such as CASH, FNO etc.
exchange_token	str	Exchange token of the equity or derivative as provided in instrument csv.

Output

```
{
  "ltp": {
    "NSE": {
      "CASH": {
        "23855": {
          "tsInMills": 1746174479582.0,
          "ltp": 1419.1
        }
      },
      "FNO": {
        "35241": {
          "tsInMills": 1746174479582.0,
          "ltp": 26111.10
        }
      }
    }
  }
}
```

Live Data for Equity and Derivatives - Response Fields

Name	Type	Description
tsInMillis	int	Epoch time in milliseconds.
ltp	float	The last traded price of the instrument.

Indices Live Data

Subscribe to, get, and unsubscribe from live data for indices.

```
from growapi import GrowFeed, GrowAPI

# Grow API Credentials (Replace with your actual credentials)
API_AUTH_TOKEN = "your-token"

grow = GrowAPI(API_AUTH_TOKEN)
feed = GrowFeed(grow)

def on_data_received(meta): # callback function which gets triggered when data is received
    print("Data received")
    print(feed.get_trigger_value())
```

```
# you can fetch exchange_token from instruments.csv file
instruments_list = [{"exchange": "NSE", "segment": "CASH", "exchange_token": "NIFTY"}, {"exchange": "BSE", "s
feed.subscribe_index_value(instruments_list, on_data_received=on_data_received)

# This is a blocking call. Nothing after this will run.
feed.consume()

# OR

# you can also fetch data synchronously
feed.subscribe_index_value(instruments_list)
# live data can also be continuously polled using this method
for i in range(10):
    print(feed.get_index_value())

feed.unsubscribe_index_value(instruments_list)
```

Live Index Data - Request Fields

Name	Type	Description
exchange	str	Stock Exchange
segment	str	Segment of the instrument such as CASH, FNO etc.
exchange_token	str	Exchange token of the equity or derivative as provided in instrument csv.

Output

```
{
  "NSE": {
    "CASH": {
      "NIFTY": {
        "tsInMillis": 1746174582295.0,
        "value": 24386.7
      }
    }
  },
  "BSE": {
    "CASH": {
      "I": {
        "tsInMillis": 1746174582295.0,
        "value": 73386.7
      }
    }
  }
}
```

Live Index Data - Response Fields

Name	Type	Description
tsInMillis	int	Epoch time in milliseconds.
value	float	The current value of the index.

Market Depth

Subscribe and get the live data of equity, index and derivatives.

Market Depth for Equity and Derivatives

Subscribe to, get, and unsubscribe from the market depth for equities or derivatives contract.

```
from growwap1 import GrowFeed, GrowAPI

# Groww API Credentials (Replace with your actual credentials)
API_AUTH_TOKEN = "your_token"

groww = GrowwAPI(API_AUTH_TOKEN)
feed = GrowwFeed(groww)

def on_data_received(meta): # callback function which gets triggered when data is received
    print("Data received")
    print(feed.get_market_depth())

# you can fetch exchange_token from instruments.csv file
instruments_list = [{"exchange": "NSE", "segment": "CASH", "exchange_token": "2885"}, {"exchange": "NSE", "se

feed.subscribe_market_depth(instruments_list, on_data_received=on_data_received)

# This is a blocking call. Nothing after this will run.
feed.consume()

# OR

# you can also fetch data synchronously
feed.subscribe_market_depth()
# market depth can also be continuously polled using this method
for i in range(10):
    print(feed.get_market_depth())

feed.unsubscribe_market_depth(instruments_list)
```

Market Depth - Request Fields

Name	Type	Description
exchange	str	Stock Exchange
segment	str	Segment of the instrument such as CASH, FNO etc.
exchange_token	str	Exchange token of the equity or derivative as provided in instrument csv.

Output

```
{
  "NSE": {
    "CASH": {
      "2885": {
        "tsInMillis": 1746156600.0,
        "buyBook": {
          "x2": {
            "price": 1418.7,
            "qty": 23.0
          },
          "x3": {
            "price": 1418.6,
            "qty": 206.0
          },
          "x1": {
            "price": 1418.8,
            "qty": 113.0
          },
          "x4": {
            "price": 1418.5,
```

```
        "qty": 774.0
    },
    "5": {
        "price": 1418.4,
        "qty": 1855.0
    }
},
"sellBook": {
    "price": 1
```

Market Depth - Response Fields

Name	Type	Description
tsInMills	int	Epoch time in milliseconds.
buyBook	Optional(dict[int, dict])	Aggregated buy orders showing demand with the different price levels as keys and quantities at the price levels as values.
sellBook	Optional(dict[int, dict])	Aggregated sell orders showing supply with the different price levels as keys and quantities at the price levels as values.

Order Updates

Subscribe and get the latest updates on execution of orders for both equity and derivatives.

Derivatives order updates

Subscribe to, get, and unsubscribe from derivative order updates.

```
from growwapl import GrowFeed, GrowAPI

# Groww API Credentials (Replace with your actual credentials)
API_AUTH_TOKEN = "your_token"

groww = GrowwAPI(API_AUTH_TOKEN)
feed = GrowwFeed(groww)

def on_data_received(meta): # callback function which gets triggered when data is received
    if(meta["feed_type"] == "order_updates" and meta["segment"] == groww.SEGMENT_FNO):
        print(feed.get_fno_order_update())

feed.subscribe_fno_order_updates(on_data_received=on_data_received)

# This is a blocking call. Nothing after this will run.
feed.consume()

# OR
# you can also fetch data synchronously
feed.subscribe_fno_order_updates()
# order update can also be continuously polled using this method
for i in range(10):
    print(feed.get_fno_order_update())

feed.unsubscribe_fno_order_updates()
```

Output

```
{
  "qty": 75,
  "price": "118",
  "filledQty": 75,
  "avgFillPrice": "118",
  "growwOrderId": "GMKFO258214158557MHR6E3F2HSE",
  "exchangeOrderId": "14888888179694433",
  "orderStatus": "EXECUTED",
  "duration": "DAY",
  "exchange": "NSE",
  "segment": "FNO",
  "product": "NRML",
  "contractId": "NIFTY2522025488CE"
}
```

Equity order updates

```
from growwapl import GrowFeed, GrowAPI

# Groww API Credentials (Replace with your actual credentials)
API_AUTH_TOKEN = "your_token"

groww = GrowwAPI(API_AUTH_TOKEN)
feed = GrowwFeed(groww)

def on_data_received(meta): # callback function which gets triggered when data is received
    if(meta["feed_type"] == "order_updates" and meta["segment"] == groww.SEGMENT_CASH):
        print(feed.get_equity_order_update())

feed.subscribe_equity_order_updates(on_data_received=on_data_received)

# This is a blocking call. Nothing after this will run.
feed.consume()

# OR
# you can also fetch data synchronously
feed.subscribe_equity_order_updates()
# order update can also be continuously polled using this method
for i in range(10):
    print(feed.get_equity_order_update())

feed.unsubscribe_equity_order_updates()
```

Output

```
{
  "qty": 3,
  "filledQty": 3,
  "avgFillPrice": "145",
  "growwOrderId": "GMK258582123553ZM58K6VX9LM",
  "exchangeOrderId": "1188888851248116",
  "orderStatus": "EXECUTED",
  "duration": "DAY",
  "exchange": "NSE",
  "contractId": "INE221H01819"
}
```

Order updates - Response Fields

Name	Type	Description
qty	int	Quantity of the equity or derivative
filledQty	int	Quantity for which trades has been executed
avgFillPrice	str	Avg price of the order placed
growwOrderId	str	Order id generated by Groww for an order
exchangeOrderId	str	Order ID assigned by the exchange for tracking purposes.

Name	Type	Description
orderStatus	str	Current status of the placed order
duration	str	Validity of the order
exchange	str	Stock Exchange
contractId	str	ISIN (International Securities Identification number) for stocks and contract symbol for derivatives

Position Updates

Subscribe and get the latest updates on creation and execution of derivatives positions.

Derivatives position updates

Subscribe to, get, and unsubscribe from position updates on your holdings.

```
from growwapl import GrowFeed, GrowAPI

# Groww API Credentials (Replace with your actual credentials)
API_AUTH_TOKEN = "your_token"

groww = GrowwAPI(API_AUTH_TOKEN)
feed = GrowwFeed(groww)

def on_data_received(meta): # callback function which gets triggered when data is received
    print(feed.get_fno_position_update())

feed.subscribe_fno_position_updates(on_data_received=on_data_received)

# This is a blocking call. Nothing after this will run.
feed.consume()

# OR
# you can also fetch data synchronously
feed.subscribe_fno_position_updates()
# position update can also be continuously polled using this method
for i in range(10):
    print(feed.get_fno_position_update())

feed.unsubscribe_fno_position_updates()
```

Output

```
{
  "symbol[isin]": "NIFTY255002400NCE",
  "exchangePosition": {
    "BSE": {},
    "NSE": {
      "creditQty": 300.0,
      "creditPrice": 3555.0,
      "debitQty": 75.0,
      "debitPrice": 5475.0
    }
  }
}
```

Position updates - Response Fields

Name	Type	Description
symbolisin	str	ISIN (International Securities Identification number) symbol of the instrument
exchangePosition	object	Contains exchange-wise position details
exchangePosition.BSE	object	Position details on BSE
exchangePosition.NSE	object	Position details on NSE
exchangePosition.NSE.creditQty	float	Quantity credited on NSE
exchangePosition.NSE.creditPrice	float	Price at which credit occurred on NSE
exchangePosition.NSE.debitQty	float	Quantity debited on NSE
exchangePosition.NSE.debitPrice	float	Price at which debit occurred on NSE

Metadata

Metadata refers to additional information about the feed data, such as the exchange, segment, feed type, and feed key. This metadata is useful for identifying and categorizing the data received from the Groww Feed.

Metadata Fields

Name	Type	Description
exchange	str	Stock Exchange
segment	str	Segment of the instrument such as CASH, FNO etc.
feed_type	str	The type of feed data (e.g., lp , order_updates , position_updates).
feed_key (Exchange token)	str	A unique identifier for the feed topic.

Example Metadata Usage

The metadata is passed to the callback function when data is received. You can use it to filter or process the data based on its attributes.

```
def on_data_received(meta):
    print("Metadata received:")
    print(meta)
    # ....
```